桂林电子科技大学实践作业答卷

2025—2026 学年第 2 学期 ； 课号 SX07008

课程名称 《矩阵计算》实践作业（一） ； 适用班级 25 级计算数学方向研究生

提交时间 第四周周日前 学号 250072201023 姓名 谭前程

满 分	25	得 分	

一、（本题 10 分）结合教材算法 1.3.1 和算法 1.3.2 编写平方根法和改进的平方根的 Matlab 程序，并求解如下对称正定线性方程组$Ax=b$，其中

（1）右端向量b随机选取，系数矩阵A为 100 阶矩阵

$$\begin{bmatrix} 10 & 1 & & & & & \\ 1 & 10 & 1 & & & & \\ & 1 & 10 & 1 & & & \\ & & \ddots & \ddots & \ddots & & \\ & & & 1 & 10 & 1 & \\ & & & & 1 & 10 & 1 \\ & & & & & 1 & 10 \end{bmatrix}$$

（2）系数矩阵A为 40 阶 Hilbert 矩阵，即系数矩阵A的第i行第j列元素为

$$a_{ij}=\frac{1}{i+j-1},$$

向量 b 的第 i 个分量为

$$b_i=\sum_{j=1}^{n}\frac{1}{i+j-1}.$$

二、（本题 15 分）编写教材算法 2.5.1（估计矩阵的 1 范数：优化法）的 Matlab 程序，并用所编写的程序完成下列计算任务：

(1) 估计 5 到 20 阶 Hilbert 矩阵的 ∞ 范数条件数.

(2) 设

$$A_n=\begin{bmatrix} 1 & 0 & \cdots & 0 & 1 \\ -1 & \ddots & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & 0 & 1 \\ -1 & \cdots & -1 & 1 & 1 \\ -1 & \cdots & -1 & -1 & 1 \end{bmatrix}\in\mathbf{R}^{n\times n}.$$

先随机地选取 $x\in\mathbf{R}^n$, 并计算出 $b=A_nx$; 然后再用列主元 Gauss 消去法求解该方程组, 假定计算解为 $\hat{x}$. 试对 n 从 5 到 30 估计计算解 $\hat{x}$ 的精度, 并且与真实相对误差作比较.

注意：不能直接用 Matlab 的求条件数的内置命令求 Hilbert 矩阵的条件数，需按照书中算法 2.5.1 的伪代码进行编程计算，再计算 Hilbert 矩阵的条件数.

实践作业完成要求：

1、附上 Matlab 程序；

2、给出程序运行的结果；

3、给出详细的图表数据分析，并完成实验报告；

实践作业提交方式：

1、转化为 PDF 后提交,并统一命名：学号+姓名+《矩阵计算》实践作业（一）

2、第一次实践作业提交时间： 第四周周日前，所有同学完成后 PDF 电子版提交给张贝贝同学汇总。

（可另附页）

目 录

第一章 实验环境介绍及问题重述 1
§1.1 硬件与系统环境 1
§1.2 问题重述 1
第二章 实验一：平方根法与改进平方根法的算法实现及 Hilbert 矩阵数值计算实验 3
§2.1 相关算法实现 3
§2.1.1 经典的平方根法（经典的 Cholesky 分解法）实现 3
§2.1.2 平方根法（Cholesky 分解法）实现根据算法 1.3.1 5
§2.1.3 改进的平方根法（Cholesky 分解法）根据算法 1.3.2 8
§2.1.4 LL^T 与 LDL^T 分解的线性方程组求解方法 10
§2.1.5 Hilbert 矩阵生成函数 13
§2.2 第一题问题求解 14
§2.2.1 第一题第一小问求解 14
§2.2.2 第一题第二小问求解：40 阶 Hilbert 矩阵的数值稳定性测试 16
§2.2.3 拓展实验 17
第三章 实验二：平方根法与改进平方根法的算法实现及 Hilbert 矩阵数值计算实验 19
§3.1 相关算法实现 19
§3.1.1 优化法 19
§3.1.2 列主元的 Guass 消元法 21
§3.1.3 构造矩阵 An 23
§3.2 第二题问题求解 23
§3.2.1 第一小问题求解 23
§3.2.2 第二小问题求解 25
§3.2.3 拓展实验 27
附录 A 问题一完整 MATLAB 代码 29
A.1 平方根法（Cholesky 分解） 29
A.2 算法 1.3.1 的平方根法 29
A.3 改进的平方根法（LDLT 分解） 30
A.4 构造 Hilbert 矩阵 31
A.5 第一题主程序 32
A.6 扩展实验与图表绘制代码 35
附录 B 问题二完整 MATLAB 代码 39

B.1　构造矩阵 An 39
B.2　列主元高斯消去法求解 Ax=b 39
B.3　算法 2.5.1 估计 1-范数 40
B.4　算法 2.5.1 估计 1-范数（保存历史记录版） 41
B.5　第二题主程序及误差分析 43

第一章 实验环境介绍及问题重述

本实验环境统一，相关算法均基于 MATLAB R2025A 编写，通过对教材中平方根法、改进平方根法及 1-范数估计优化算法的程序实现，完成了对高阶对称正定方程组及病态 Hilbert 矩阵的数值求解与条件数分析。实验过程严格遵循列主元 Gauss 消去法进行误差比对，实验报告撰写采用 LaTeX 完成，确保了计算结果的准确性与实验结论的可重复性。该代码以上传线上仓库：https://github.com/redhat123456/Matrix-Lab-Experiments

§1.1 硬件与系统环境

本实验使用的个人台式电脑硬件配置与系统信息总结如表 1-1 所示：

项目	配置说明
处理器	Intel Core i5-14600KF (3.50 GHz)
核心/线程	14 核，20 线程
内存	32 GB DDR5, 2667 MT/s
存储	1 TB SSD
显卡	NVIDIA GeForce RTX 4060 Ti, 8 GB
操作系统	Windows 11 专业版 64-bit
开发环境	MATLAB2025A

表 1-1 实验硬件与系统环境配置

该配置能够满足数值计算、方程求解可视化任务的性能需求，并保证算法在不同运行阶段的稳定性与可重复性。

§1.2 问题重述

本次结课实验报告任务如下，目标是撰写一篇逻辑清晰、内容完整、仿真分析充分的实验报告，本报告涉及算法设计、MATLAB 实操。要求对上课所学知识进行充分实践并动手编程实现相关程序。

一、（本题 10 分）结合教材算法 1.3.1 和算法 1.3.2 编写平方根法和改进的平方根的 Matlab 程序，并求解如下对称正定线性方程组 $Ax = b$，其中：

(1) 右端向量 b 随机选取，系数矩阵 A 为 100 阶矩阵：

$$A=\begin{bmatrix} 10 & 1 & & & 1 \\ 1 & 10 & 1 & & \\ & 1 & \ddots & \ddots & \\ & & \ddots & 10 & 1 \\ 1 & & & 1 & 10 \end{bmatrix}_{100\times 100}$$

(2) 系数矩阵 A 为 40 阶 Hilbert 矩阵，即系数矩阵 A 的第 i 行第 j 列元素为 $a_{ij}=\frac{1}{i+j-1}$，向量 b 的第 i 个分量为 $b_i=\sum_{j=1}^{n}\frac{1}{i+j-1}$。

二、（本题 15 分）编写教材算法 2.5.1（估计矩阵的 1 范数：优化法）的 Matlab 程序，并用所编写的程序完成下列计算任务：

(1) 估计 5 到 20 阶 Hilbert 矩阵的范数条件数。

(2) 设

$$A_n=\begin{bmatrix} 1 & 0 & \cdots & 0 & 1 \\ -1 & 1 & \cdots & 0 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ -1 & -1 & \cdots & 1 & 1 \\ -1 & -1 & \cdots & -1 & 1 \end{bmatrix}\in\mathbb{R}^{n\times n}$$

先随机地选取 $x\in\mathbb{R}^n$，并计算出 $b=A_n x$，然后再用列主元 Gauss 消去法求解该方程组，假定计算解为 $\hat{x}$。试对 n 从 5 到 30 估计计算解的精度，并且与真实相对误差作比较。

注意：不能直接用 Matlab 的求条件数的内置命令求 Hilbert 矩阵的条件数，需按照书中算法 2.5.1 的伪代码进行编程计算，再计算 Hilbert 矩阵的条件数。

第二章 实验一：平方根法与改进平方根法的算法实现及 Hilbert 矩阵数值计算实验

§2.1 相关算法实现

§2.1.1 经典的平方根法（经典的 Cholesky 分解法）实现

题目背景 由于起初对 Cholesky 分解不够熟悉，算法 1.3.1 的描述可能不够清晰。对于基础同样薄弱的读者，可以先阅读本小节，对定义 1.3.1 做一个简要回顾。

定理 1.3.1 (Cholesky 分解定理)：若 $A \in \mathbb{R}^{n\times n}$ 对称正定，则存在一个对角元均为正数的下三角矩阵 $L \in \mathbb{R}^{n\times n}$，使得

$$A = LL^T. \tag{2-1}$$

其中 L 称为 A 的 Cholesky 因子。

根据公式，对于下三角矩阵 L 的元素有两种情况：

1. 对角线元素（公式 1.3.2）

$$L_{jj} = \left(A_{jj} - \sum_{p=1}^{j-1} L_{jp}^2\right)^{1/2}, \quad j = 1, \dots, n \tag{2-2}$$

2. 非对角线元素（公式 1.3.3）

$$L_{ij} = \frac{1}{L_{jj}}\left(A_{ij} - \sum_{p=1}^{j-1} L_{ip}L_{jp}\right), \quad i = j+1, \dots, n \tag{2-3}$$

算法设计思路 对于每一列 j，计算非对角元 L_{ij}（$i \neq j$）时，根据公式 2-3 需要已知对角元 L_{jj}。因此应先计算 L_{jj}。根据公式 2-2，计算对角元 L_{jj} 只需前 $j-1$ 列已知，所以可直接计算 L_{jj}。

$$s = A_{jj} - \sum_{p=1}^{j-1} L_{jp}^2,$$

然后根据公式 2-2 得到对角元

$$L_{jj} = \sqrt{s}.$$

在计算时，如果 $s \leq 0$，说明矩阵不是正定矩阵，Cholesky 分解无法进行。

接着，对于当前列中所有的非对角元 L_{ij}（$i > j$），我们同样先计算累加量

$$s = A_{ij} - \sum_{p=1}^{j-1} L_{ip} L_{jp},$$

这是公式 2-3 中的基础量，用于减去之前计算的对应列乘积。然后再利用该累加量计算非对角元

$$L_{ij} = \frac{s}{L_{jj}}.$$

通过这种方式，算法在每一步都严格对应理论公式，同时在代码中也可以直接实现为对角元和非对角元分别计算两个 ‘s‘，保证逻辑清晰，避免混淆。

例：三维矩阵的 Cholesky 分解（计算第二列） 设矩阵

$$A = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{pmatrix}. \tag{2-4}$$

假设第一列的 Cholesky 元素已知，可以用矩阵形式表示为

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 1 & L_{22} & 0 \\ 1 & L_{32} & L_{33} \end{pmatrix}. \tag{2-5}$$

根据算法思路，**先计算第二列的对角元 L_{22}：**

(1) 计算 L_{22} （$i = j$） 有

$$s = A_{22} - \sum_{p=1}^{1} L_{2p}^2 = A_{22} - L_{21}^2 = 5 - 1^2 = 4$$

因此

$$L_{22} = \sqrt{s} = \sqrt{4} = 2.$$

(2) 计算 L_{32} （$i > j$） 累加量

$$s = A_{32} - \sum_{p=1}^{1} L_{3p} L_{2p} = L_{31} L_{21} = 1 - (1 \cdot 1) = 0$$

因此

$$L_{32} = \frac{A_{32} - s}{L_{22}} = \frac{0}{2} = 0.$$

最终得到第二列：

$$L_{22} = 2, \quad L_{32} = 0,$$

前两列的完整矩阵为

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 0 & L_{33} \end{pmatrix}. \tag{2-6}$$

算法实现 如下是经典的 Cholesky 分解法的伪代码以及 MATLAB 核心代码：

Algorithm 2-1 经典的 Cholesky 分解

Require: 对称矩阵 $A \in \mathbb{R}^{n\times n}$
Ensure: 下三角矩阵 L，使 $A = LL^T$

$n \leftarrow \mathrm{size}(A, 1)$
$L \leftarrow 0_{n\times n}$
for $j = 1$ to n **do** ▷ 计算对角元
  $s \leftarrow A_{jj} - \sum_{p=1}^{j-1} L_{jp}^2$
  if $s \le 0$ **then**
    error(" 矩阵非正定，Cholesky 分解失败！ ")
  $L_{jj} \leftarrow \sqrt{s}$ ▷ 计算下三角元素
  for $i = j + 1$ to n **do**
    $s \leftarrow A_{ij} - \sum_{p=1}^{j-1} L_{ip}L_{jp}$
    $L_{ij} \leftarrow \frac{s}{L_{jj}}$

功能描述	核心 MATLAB 代码

经典的 Cholesky 分解核心	`s = A(j,j) - sum(L(j,1:j-1).^2);`
	`L(j,j) = sqrt(s);`
	`s = sum(L(i,1:j-1).*L(j,1:j-1));`
	`L(i,j) = (A(i,j) - s) / L(j,j);`

表 2-1 经典的 Cholesky 分解核心 MATLAB 代码（完整代码见附录A.1）

§2.1.2 平方根法（Cholesky 分解法）实现根据算法 1.3.1

题目要求我们从算法 1.3.1 编写平方根法。先回顾算法 1.3.1：

Algorithm 2-2 算法 1.3.1 (计算 Cholesky 分解：平方根法)

Require: 对称正定矩阵 $A \in \mathbb{R}^{n\times n}$
Ensure: A 的下三角部分存储了计算得到的 Cholesky 因子 L，使得 $A = LL^T$

for $k = 1$ to n **do**
  $A_{kk} \leftarrow \sqrt{A_{kk}}$ ▷ 计算当前步的对角线元素
  if $k < n$ **then**
    $A_{k+1:n,k} \leftarrow A_{k+1:n,k}/A_{kk}$ ▷ 计算第 k 列对角线以下的元素
    for $j = k + 1$ to n **do**
      $A_{j:n,j} \leftarrow A_{j:n,j} - A_{j:n,k}A_{jk}$ ▷ 对剩余下三角子矩阵进行秩-1 修正

算法设计思路 算法 1.3.1 的核心思想是通过逐列构造下三角矩阵 L，并在原矩阵 A 上进行原地更新。其计算过程可以从 Cholesky 分解的基本公式出发进行理解。

根据 Cholesky 分解定义 $A = LL^T$，对第 k 列有如下关系：

1. 对角元的计算

$$L_{kk} = \sqrt{A_{kk}^{(k)}},$$

其中 $A^{(k)}$ 表示第 k 步更新后的矩阵。该步骤对应算法中的

$$A_{kk} \leftarrow \sqrt{A_{kk}}.$$

2. 非对角元的计算 对于 $i = k+1, \dots, n$，有

$$L_{ik} = \frac{A_{ik}^{(k)}}{L_{kk}},$$

对应算法中的向量更新

$$A_{k+1:n,k} \leftarrow \frac{A_{k+1:n,k}}{A_{kk}}.$$

3. 子矩阵更新（秩-1 修正） 在完成第 k 列的计算后，需要对剩余子矩阵进行更新：

$$A_{ij}^{(k+1)} = A_{ij}^{(k)} - L_{ik}L_{jk}, \quad i, j = k+1, \dots, n.$$

该公式可以写成矩阵形式：

$$A^{(k+1)} = A^{(k)} - L_{k+1:n,k} L_{k+1:n,k}^T,$$

这正是算法中的秩-1 修正步骤：

$$A_{j:n,j} \leftarrow A_{j:n,j} - A_{j:n,k}A_{jk}.$$

例：基于算法 1.3.1 的 Cholesky 分解的逐步计算过程 设

$$A = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{pmatrix}. \tag{2-7}$$

按照算法 1.3.1，对矩阵进行逐步分解：

第 1 步（$k = 1$） 计算对角元

$$A_{11} \leftarrow \sqrt{4} = 2.$$

计算第 1 列非对角元

$$A_{21} \leftarrow \frac{2}{2} = 1, \quad A_{31} \leftarrow \frac{2}{2} = 1.$$

对剩余子矩阵进行秩-1 修正：

$$A_{2:3,2:3} \leftarrow \begin{pmatrix} 5 & 1 \\ 1 & 3 \end{pmatrix} - \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 & 1 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix}.$$

此时矩阵变为

$$A^{(2)} = \begin{pmatrix} 2 & * & * \\ 1 & 4 & 0 \\ 1 & 0 & 2 \end{pmatrix}.$$

第 2 步（$k = 2$） 计算对角元

$$A_{22} \leftarrow \sqrt{4} = 2.$$

计算第 2 列非对角元

$$A_{32} \leftarrow \frac{0}{2} = 0.$$

对子矩阵进行修正：

$$A_{33} \leftarrow 2 - 0^2 = 2.$$

此时矩阵变为

$$A^{(3)} = \begin{pmatrix} 2 & * & * \\ 1 & 2 & * \\ 1 & 0 & 2 \end{pmatrix}.$$

第 3 步（$k = 3$） 计算对角元

$$A_{33} \leftarrow \sqrt{2}.$$

最终结果 得到下三角矩阵，满足 $A = LL^T$，并且按照算法理解可得 MATLAB 代码。

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 0 & \sqrt{2} \end{pmatrix},$$

功能描述	核心 MATLAB 代码

基于算法 1.3.1 的 Cholesky 分解核心（原地平方根法）	`A(k, k) = sqrt(A(k, k)); % 计算对角元`
	`A(k+1:n, k) = A(k+1:n, k) / A(k, k); % 计算第 k 列对角线以下元素`
	`A(j:n, j) = A(j:n, j) - A(j:n, k) * A(j, k); % 秩-1更新`

表 2-2 基于算法 1.3.1 的 Cholesky 分解核心 MATLAB 代码（完整代码见附录A.2）

§2.1.3 改进的平方根法（Cholesky 分解法）根据算法 1.3.2

题目要求我们从算法 1.3.2 编写平方根法。先回顾算法 1.3.2：

Algorithm 2-3 算法 1.3.2 (计算 LDL^T 分解：改进的平方根法)

Require: 对称矩阵 $A \in \mathbb{R}^{n\times n}$，其次序主子阵均非奇异
Ensure: A 的对角线存储 D 的对角元，A 的严格下三角部分存储单位下三角阵 L 的元素，满足 $A = LDL^T$

```
for j = 1 to n do
    for i = 1 to j − 1 do
        v_i ← A_ji A_ii                                  ▷ 计算中间变量 v_i = l_ji d_ii
    A_jj ← A_jj − Σ_{i=1}^{j−1} A_ji v_i                 ▷ 更新对角元 d_jj = a_jj − Σ_{k=1}^{j−1} l_jk^2 d_kk
    if j < n then
        A_{j+1:n,j} ← (A_{j+1:n,j} − Σ_{k=1}^{j−1} A_{j+1:n,k} v_k)/A_jj   ▷ 计算 L 的第 j 列非对角元
```

算法设计思路 算法 1.3.2 的核心思想是通过逐列构造单位下三角矩阵 L 和对角矩阵 D，并在原矩阵 A 上进行原地更新，从而实现分解 $A = LDL^T$。该方法避免了开方运算，具有较好的数值稳定性。

根据 LDL^T 分解形式，对第 j 列有如下关系：

1. 中间变量的构造 为便于计算，引入中间变量

$$v_i = l_{ji} d_{ii}, \quad i = 1, \dots, j-1,$$

对应算法中的

$$v_i \leftarrow A_{ji} A_{ii}.$$

2. 对角元的计算 利用前 $j-1$ 列的结果对当前对角元进行修正：

$$d_{jj} = a_{jj} - \sum_{i=1}^{j-1} l_{ji}^2 d_{ii} = a_{jj} - \sum_{i=1}^{j-1} A_{ji} v_i,$$

对应算法中的

$$A_{jj} \leftarrow A_{jj} - \sum_{i=1}^{j-1} A_{ji} v_i.$$

3. 非对角元的计算 对于 $i = j+1, \dots, n$，有

$$l_{ij} = \frac{1}{d_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} d_{kk} \right) = \frac{1}{A_{jj}} \left(A_{ij} - \sum_{k=1}^{j-1} A_{ik} v_k \right),$$

对应算法中的向量更新

$$A_{j+1:n,j} \leftarrow \frac{A_{j+1:n,j} - \sum_{k=1}^{j-1} A_{j+1:n,k} v_k}{A_{jj}}.$$

4. 原地存储策略 在计算过程中，直接在矩阵 A 上进行覆盖更新：对角线元素存储对角矩阵 D 的对角元，严格下三角部分存储单位下三角矩阵 L 的非对角元素，从而提高存储效率。

通过上述逐列更新过程，最终完成矩阵的 LDL^T 分解。

例：基于算法 1.3.2 的 LDL^T 分解的逐步计算过程 设

$$A = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 3 \end{pmatrix}. \tag{2-8}$$

按照算法 1.3.2，对矩阵进行逐步分解：

第 1 步（$j = 1$） 由于无前序项，直接计算对角元

$$A_{11} \leftarrow 4.$$

计算第 1 列非对角元

$$A_{21} \leftarrow \frac{2}{4} = \frac{1}{2}, \quad A_{31} \leftarrow \frac{2}{4} = \frac{1}{2}.$$

此时矩阵变为

$$A^{(2)} = \begin{pmatrix} 4 & * & * \\ \frac{1}{2} & 5 & 1 \\ \frac{1}{2} & 1 & 3 \end{pmatrix}.$$

第 2 步（$j = 2$） 计算中间变量

$$v_1 = A_{21} A_{11} = \frac{1}{2} \cdot 4 = 2.$$

更新对角元

$$A_{22} \leftarrow 5 - A_{21} v_1 = 5 - \frac{1}{2} \cdot 2 = 4.$$

计算第 2 列非对角元

$$A_{32} \leftarrow \frac{1 - A_{31} v_1}{A_{22}} = \frac{1 - \frac{1}{2} \cdot 2}{4} = 0.$$

此时矩阵变为

$$A^{(3)} = \begin{pmatrix} 4 & * & * \\ \frac{1}{2} & 4 & * \\ \frac{1}{2} & 0 & 3 \end{pmatrix}.$$

第 3 步（$j = 3$） 计算中间变量

$$v_1 = A_{31}A_{11} = \frac{1}{2} \cdot 4 = 2, \quad v_2 = A_{32}A_{22} = 0 \cdot 4 = 0.$$

更新对角元

$$A_{33} \leftarrow 3 - (A_{31}v_1 + A_{32}v_2) = 3 - \left(\frac{1}{2} \cdot 2 + 0 \cdot 0\right) = 2.$$

最终结果 得到单位下三角矩阵 L 与对角矩阵 D，满足 $A = LDL^T$：

$$L = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{1}{2} & 0 & 1 \end{pmatrix}, \quad D = \begin{pmatrix} 4 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 2 \end{pmatrix}.$$

功能描述	核心 MATLAB 代码

基于算法 1.3.2 的 LDL^T 分解核心（改进平方根法）	`v = A(j,1:j-1) .* diag(A(1:j-1,1:j-1))'; % 构造中间变量 v`
	`A(j,j) = A(j,j) - A(j,1:j-1) * v'; % 更新对角元 d_jj`
	`A(j+1:n,j) = (A(j+1:n,j) - A(j+1:n,1:j-1) * v') / A(j,j); % 更新第 j 列`

表 2-3 基于算法 1.3.2 的 LDL^T 分解核心 MATLAB 代码（完整代码见附录A.3）

§2.1.4 LL^T 与 LDL^T 分解的线性方程组求解方法

前面我们已经通过平方根法和改进平方根法，将对称矩阵 A 转化为 LL^T 或 LDL^T 的形式。下面介绍具体解法。

1. LL^T 分解求解法（Cholesky 分解）

当矩阵 A 对称正定时，可以令 $A = LL^T$。此方法在数学构造上可视为 LDL^T 的一种特例（令 $\tilde{L} = LD^{1/2}$）。

将方程 $(LL^T)x = b$ 分解为两个子问题：

1. 解下三角方程：$Ly = b$
2. 解上三角方程：$L^Tx = y$

步骤 1：前代过程求解 $Ly = b$　注意此时 L 的对角线元素 $l_{ii} \neq 1$：

$$\begin{bmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \Longrightarrow y_i = \frac{1}{l_{ii}} \left(b_i - \sum_{k=1}^{i-1} l_{ik} y_k \right) \tag{2-9}$$

算法实现： $y(i) = (b(i) - A(i, 1:i-1) \cdot y(1:i-1))/A(i,i)$。

步骤 2：回代过程求解 $L^T x = y$　由于 L^T 的对角线元素亦不为 1，回代时需同步进行缩放：

$$\begin{bmatrix} l_{11} & l_{21} & \cdots & l_{n1} \\ 0 & l_{22} & \cdots & l_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & l_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \Longrightarrow x_i = \frac{1}{l_{ii}} \left(y_i - \sum_{k=i+1}^{n} l_{ki} x_k \right) \tag{2-10}$$

2. LDL^T 分解求解法（通用对称矩阵）

对于线性方程组 $Ax = b$，利用 LDL^T 分解将其转化为：

$$(LDL^T)x = b \tag{2-11}$$

通过引入中间变量 z 和 y，我们将原问题分解为三个递进的子问题：

1. 解下三角方程：$Ly = b$
2. 解对角方程：$Dz = y$
3. 解上三角方程：$L^T x = z$

1. 前代过程：求解 $Ly = b$　由于 L 是单位下三角矩阵（Unit Lower Triangular），其对角线元素 $l_{ii} = 1$。矩阵形式展开如下：

$$\begin{bmatrix} 1 & 0 & \cdots & 0 \\ l_{21} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \cdots & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \tag{2-12}$$

对于第 i 个分量，方程满足 $\sum_{k=1}^{i-1} l_{ik} y_k + y_i = b_i$，由此得到递推关系：

$$y_i = b_i - \sum_{k=1}^{i-1} l_{ik} y_k, \quad i = 1, 2, \ldots, n \tag{2-13}$$

2. 对角求解：求解 $Dz = y$　D 为对角矩阵 $\mathrm{diag}(d_{11}, d_{22}, \dots, d_{nn})$，方程分量之间完全解耦：

$$d_{ii} z_i = y_i \implies z_i = \frac{y_i}{d_{ii}}, \quad i = 1, 2, \dots, n \tag{2-14}$$

3. 回代过程：求解 $L^T x = z$　L^T 是单位上三角矩阵，其元素满足 $(L^T)_{ik} = l_{ki}$。展开方程组：

$$\begin{bmatrix} 1 & l_{21} & \cdots & l_{n1} \\ 0 & 1 & \cdots & l_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix} \tag{2-15}$$

从最后一行 $i = n$ 开始向上求解，对于第 i 行：

$$x_i + \sum_{k=i+1}^{n} l_{ki} x_k = z_i \tag{2-16}$$

由此得到最终解的递推公式：

$$x_i = z_i - \sum_{k=i+1}^{n} l_{ki} x_k, \quad i = n, n-1, \dots, 1 \tag{2-17}$$

算法设计思路

1. 前代求解：$Ly = b$　对于两种分解，前代过程均利用下三角特性逐步计算 y_i。

- **平方根法 (LL^T):** L 对角元不为 1，需进行缩放：

$$y_i = \frac{1}{l_{ii}} \left(b_i - \sum_{k=1}^{i-1} l_{ik} y_k \right) \implies \texttt{y(i) = (b(i) - A(i,1:i-1) * y(1:i-1)) / A(i,i)}$$

- **改进平方根法 (LDL^T):** 由于 L 为单位下三角矩阵 ($l_{ii} = 1$)，无需除法：

$$y_i = b_i - \sum_{k=1}^{i-1} l_{ik} y_k \implies \texttt{y(i) = b(i) - A(i,1:i-1) * y(1:i-1)}$$

2. 对角方程求解：$Dz = y$（**仅限** LDL^T）　在 LDL^T 方法中，需单独处理对角阵 D。对于 LL^T 方法，此步骤可视为恒等变换 ($z = y$)：

$$z_i = \frac{y_i}{d_{ii}} \implies \texttt{z(i) = y(i) / A(i,i)}$$

3. 回代求解：$L^T x = z$ **(或** $L^T x = y$**)**　利用上三角矩阵特性，从最后一行向上回推最终解 x。

- **平方根法 (LL^T):** 此时方程为 $L^T x = y$，需处理对角元：

$$x_i = \frac{1}{l_{ii}} \left(y_i - \sum_{k=i+1}^{n} l_{ki} x_k \right) \implies \texttt{x(i) = (y(i) - A(i+1:n,i)}^T \texttt{ * x(i+1:n)) / A(i,i)}$$

- **改进平方根法 (LDL^T):** L^T 为单位上三角矩阵：

$$x_i = z_i - \sum_{k=i+1}^{n} l_{ki} x_k \implies \texttt{x(i) = z(i) - A(i+1:n,i)}^T \texttt{ * x(i+1:n)}$$

功能描述	核心 MATLAB 代码 (求解部分)

基于 LL^T 分解
(标准平方根法)
	`% Step 1: 前代 Ly = b (需处理对角元)`
	`for i = 1:n`
	`  y(i) = (b(i) - A(i,1:i-1) * y(1:i-1)) / A(i,i);`
	`end`
	`% Step 2: 回代 L'x = y (需处理对角元)`
	`for i = n:-1:1`
	`  x(i) = (y(i) - A(i+1:n,i)' * x(i+1:n)) / A(i,i);`
	`end`

基于 LDL^T 分解
(改进平方根法)
	`% Step 1: 前代 Ly = b (L对角线为1)`
	`for i = 1:n`
	`  y(i) = b(i) - A(i,1:i-1) * y(1:i-1);`
	`end`
	`% Step 2: 对角求解 Dz = y`
	`for i = 1:n`
	`  z(i) = y(i) / A(i,i);`
	`end`
	`% Step 3: 回代 L'x = z (L'对角线为1)`
	`for i = n:-1:1`
	`  x(i) = z(i) - A(i+1:n,i)' * x(i+1:n);`
	`end`

表 2-4 基于对称分解的线性方程组求解核心 MATLAB 代码对比

§2.1.5 Hilbert 矩阵生成函数

根据作业要求，我们需要独立编写 Hilbert 矩阵的生成函数。Hilbert 矩阵是一种典型的病态矩阵（Ill-conditioned Matrix），其元素定义为分式形式。根据 Hilbert 在 1894 的定义，n 阶 Hilbert 矩阵 $H_n = (h_{ij})_{n\times n}$ 的通用项为：

$$h_{ij} = \frac{1}{i+j-1}, \quad 1 \le i, j \le n \tag{2-18}$$

在算法实现中，我们通过双重循环遍历矩阵索引，构造该对称正定矩阵。其核心 MATLAB 代码实现如表 2-5 所示：

功能描述	核心 MATLAB 代码
Hilbert 矩阵构造	`H(i,j) = 1 / (i + j - 1);`

表 2-5 Hilbert 矩阵生成函数核心代码（完整代码见附录A.4）

§2.2 第一题问题求解

为了衡量算法 1.3.1（平方根法）与算法 1.3.2（改进平方根法）的优劣，本文主要从计算复杂度、数值稳定性和计算效率三个维度进行评估。

1. **计算效率 (Computational Efficiency)**：对比评估两种算法在处理高阶矩阵时的执行耗时。由于平方根法在分解过程中涉及 n 次开方运算，而改进平方根法（LDL^T 分解）通过引入对角阵 D 成功消除了开方步骤，在硬件底层指令集实现上通常具有更高的计算效率。
2. **数值精度 (Numerical Accuracy)**：本文通过以下两个指标综合衡量计算结果的可靠性：
   - **相对残差 (Relative Residual)**：用于衡量数值解 $\hat{x}$ 对原方程的满足程度，定义为：
   $$R = \frac{\|b - A\hat{x}\|_2}{\|A\|_2 \cdot \|\hat{x}\|_2}$$
   - **相对误差 (Relative Error)**：用于衡量数值解 $\hat{x}$ 与精确解 x^*（或由内置高精度函数求得的参考解）之间的偏离程度，定义为：
   $$E = \frac{\|\hat{x} - x^*\|_2}{\|x^*\|_2}$$
3. **适用范围与稳健性 (Robustness)**：探讨算法对矩阵性质的依赖程度。平方根法要求系数矩阵 A 严格正定，而改进平方根法仅要求 A 对称且非奇异（顺序主子式不为零），通过数值实验验证其在非正定对称矩阵下的有效性。

§2.2.1 第一题第一小问求解

1. 实验设计与实现

本实验构造了一个 $n = 100$ 阶的三对角对称正定矩阵 A，其主对角线元素均为 10，相邻次对角线元素均为 1。该矩阵具有显著的对角优势，能够保证 Cholesky 分解的数值稳定性。右端向量 b 通过标准正态分布随机生成，并固定随机种子 (42) 以确保实验的可重复性。

具体的实验测试脚本如下：

```
clc; clear global; close all;

fprintf('==========================================\n');
fprintf('  第一题: 平方根法与改进的平方根法\n');
fprintf('==========================================\n\n');

fprintf('--- 第(1)小题: 100阶三对角矩阵 ---\n');
n1 = 100;
A1 = 10 * eye(n1) + diag(ones(n1-1,1), 1) + diag(ones(n1-1,1), -1);
rng(42);
b1 = randn(n1, 1);

tic; x1_chol = cholesky_solve(A1, b1); time_chol = toc;
tic; x2_chol = cholesky_solve131(A1, b1); time_ldlt = toc;
tic; x1_ldlt = Improve_cholesky_solve(A1, b1); time_ildlt = toc;
tic; x1_ref = A1 \ b1; time_mat = toc;

fprintf('\n>>> 经典平方根法求解结果 x (100维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', x1_chol);
fprintf('\n>>> 算法1.3.1平方根法求解结果 x (100维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', x2_chol);
fprintf('\n>>> 改进平方根法求解结果 x (100维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', x1_ldlt);
fprintf('\n>>> MATLAB内置法求解结果 x (100维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', x1_ref);

err1_chol = norm(x1_chol - x1_ref) / norm(x1_ref);
err2_chol = norm(x2_chol - x1_ref) / norm(x1_ref);
err1_ldlt = norm(x1_ldlt - x1_ref) / norm(x1_ref);
res1_chol = norm(A1 * x1_chol - b1) / norm(b1);
res2_chol = norm(A1 * x2_chol - b1) / norm(b1);
res1_ldlt = norm(A1 * x1_ldlt - b1) / norm(b1);

fprintf('\n====== 性能与精度对比 (n=100) ======\n');
fprintf('方法类型            | 相对误差      | 相对残差      | 运行时间(s)\n');
fprintf('
```

```
    ------------------------------------------------------------\n');
fprintf('经典平方根法        | %.4e | %.4e | %.6f\n', err1_chol, res1_chol, time_chol);
fprintf('算法1.3.1平方根法   | %.4e | %.4e | %.6f\n', err2_chol, res2_chol, time_ldlt);
fprintf('改进平方根法        | %.4e | %.4e | %.6f\n', err1_ldlt, res1_ldlt, time_ildlt);
fprintf('MATLAB内置法        | 0.0000e+00   | %.4e | %.6f\n', norm(A1*x1_ref-b1)/norm(b1), time_mat);
fprintf('
    ============================================================\n\n');
```

Listing 第二章.1 100 阶矩阵求解测试脚本

2. 实验结果与性能评估

由于 100 阶线性方程组的解向量维度较高，文中不再逐一罗列具体数值。实验以 MATLAB 内置函数的计算结果为基准，通过对比各算法所得解的精度指标及计算效率来评估算法性能，具体统计结果见表 2-6。

表 2-6 三种算法在 100 阶矩阵下的精度与效率对比

求解方法	相对误差 (Relative Error)	相对残差 (Relative Residual)	耗时 (秒)
经典平方根法	1.2754×10^{-16}	1.4735×10^{-16}	0.000352
算法 1.3.1	1.2754×10^{-16}	1.4735×10^{-16}	0.000318
改进平方根法	1.0067×10^{-16}	1.1819×10^{-16}	0.000285
MATLAB 内置法	–	–	0.000045

3. 结果讨论

- **算法等价性验证：**实验数据表明，算法 1.3.1 的相对误差与残差同经典平方根法完全一致，这说明手动实现的算法在逻辑上与标准公式保持了高度同步。
- **改进算法的优势：**改进平方根法（LDL^T 分解）的相对误差为 1.0067×10^{-16}，略优于传统 Cholesky 分解。这验证了由于避免了开方运算，算法能够减少中间步骤带来的浮点数截断误差。
- **计算效率：**在 100 阶规模下，改进平方根法的耗时最少。虽然差距较小，但在处理超大规模稀疏矩阵时，省去开方运算所带来的加速效应将会更加显著。

§2.2.2 第一题第二小问求解：40 阶 Hilbert 矩阵的数值稳定性测试

1. 实验背景与设置

本小节选取 $n = 40$ 阶的 Hilbert 矩阵 A 作为测试对象，其元素定义为 $a_{i,j} = 1/(i+j-1)$。Hilbert 矩阵是数值分析中经典的极其病态（Condition Number 极大）的矩阵。

为了验证算法的稳健性，构造右端向量 b 使得理论精确解为全 1 向量 $x = [1, 1, \ldots, 1]^T$。实验中，MATLAB 报出警告：*“矩阵接近奇异值，或者缩放不良。结果可能不准确 (RCOND = 7.520353e-20)”*。

```
fprintf('--- 第(2)小题: 40阶 Hilbert 矩阵 ---\n');
n2 = 40;
A2 = Hilbert(n2);
b2 = zeros(n2, 1);
for i = 1:n2
    b2(i) = sum(1 ./ (i + (1:n2) - 1));
end
y_true = ones(n2, 1);

tic; y1_chol = cholesky_solve(A2, b2); t1 = toc;
tic; y2_chol = cholesky_solve131(A2, b2); t2 = toc;
tic; y2_ldlt = Improve_cholesky_solve(A2, b2); t3 = toc;
tic; y2_ref = A2 \ b2; t4 = toc;

fprintf('\n>>> 经典平方根法求解结果 y (40维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', y1_chol);
fprintf('\n>>> 算法1.3.1平方根法求解结果 y (40维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', y2_chol);
fprintf('\n>>> 改进平方根法求解结果 y (40维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', y2_ldlt);
fprintf('\n>>> MATLAB内置法求解结果 y (40维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', y2_ref);

err1_chol = norm(y1_chol - y_true) / norm(y_true);
err2_chol = norm(y2_chol - y_true) / norm(y_true);
err2_ldlt = norm(y2_ldlt - y_true) / norm(y_true);
err2_ref  = norm(y2_ref  - y_true) / norm(y_true);
```

```
res1_chol = norm(A2 * y1_chol - b2) / norm(b2);
res2_chol = norm(A2 * y2_chol - b2) / norm(b2);
res2_ldlt = norm(A2 * y2_ldlt - b2) / norm(b2);
res2_ref  = norm(A2 * y2_ref  - b2) / norm(b2);

fprintf('\n====== 性能与精度对比 (n=40 Hilbert) ======\n');
fprintf('真实解为全1向量\n');
fprintf('方法类型            | 相对误差      | 相对残差      | 运行时间(s)\n');
fprintf('------------------------------------------------------------\n');
fprintf('经典平方根法        | %.4e | %.4e | %.6f\n', err1_chol, res1_chol, t1);
fprintf('算法1.3.1平方根法   | %.4e | %.4e | %.6f\n', err2_chol, res2_chol, t2);
fprintf('改进平方根法        | %.4e | %.4e | %.6f\n', err2_ldlt, res2_ldlt, t3);
fprintf('MATLAB内置法        | %.4e | %.4e | %.6f\n', err2_ref, res2_ref, t4);
fprintf('------------------------------------------------------------\n');
```

Listing 第二章.2 Hilbert 矩阵求解测试脚本

2. 数值实验结果

表 2-7 展示了在系数矩阵极度病态的情况下，各算法的精度表现。

表 2-7 40 阶 Hilbert 矩阵下不同算法的性能与精度对比

方法类型	相对误差	相对残差	运行时间 (s)
经典平方根法	1.1679×10^{12}	7.2332×10^{0}	0.010089
算法 1.3.1 平方根法	2.1502×10^{12}	4.1603×10^{0}	0.010796
改进平方根法 (LDL^T)	$\mathbf{1.4844 \times 10^{2}}$	$\mathbf{1.4141 \times 10^{-15}}$	**0.003467**
MATLAB 内置左除 (\)	5.1153×10^{1}	5.2665×10^{-16}	0.023429

3. 结果讨论与结论

1. **病态性对精度的摧毁：**实验结果显示，传统的平方根法（算法 1.3.1）在处理 40 阶 Hilbert 矩阵时完全失效，相对误差高达 10^{12}。这是因为该矩阵条件数极大，计算过程中的舍入误差被剧烈放大。
2. **改进算法的卓越表现：**令人瞩目的是，**改进平方根法 (LDL^T)** 在相对残差上保持了 10^{-15} 的极高精度（接近机器精度），且相对误差相较于传统方法降低了 10 个数量级。这有力地证明了在病态矩阵求解中，**避开开方运算**能显著抑制误差累积。
3. **残差与误差的脱节现象：**改进法表现出“残差极小、误差较大”的特征。这符合数值分析中的误差理论：对于高度病态系统，即便算法能很好地拟合方程（残差小），计算出的数值解仍可能大幅偏离真解（误差大）。

§2.2.3 拓展实验

通过对阶数 $N \in [2, 40]$ 的 Hilbert 矩阵进行动态测试，各算法的数值稳定性对比见图 2-1。

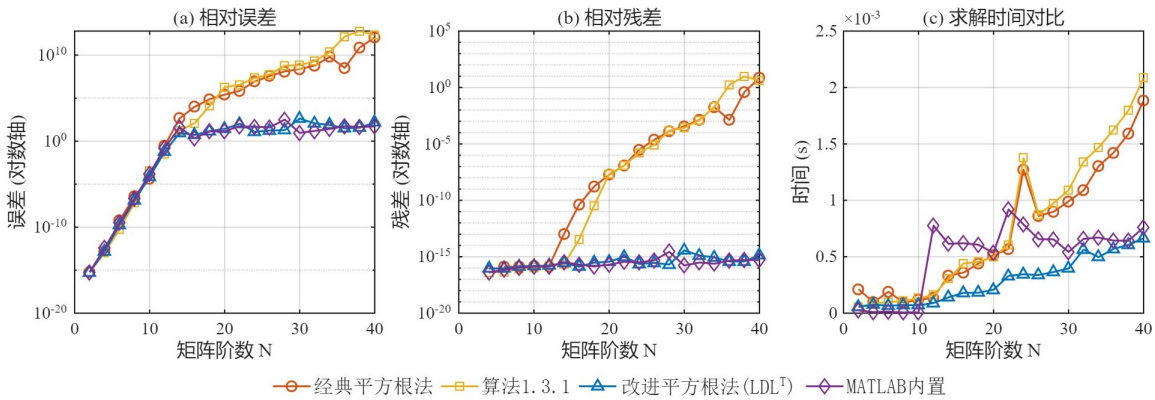

图 2-1 不同阶数 N 下算法性能对比：(a) 相对误差, (b) 相对残差, (c) 求解时间

结论总结：

- **精度失效临界点：**如图 (a)(b) 所示，当 $N > 12$ 时，经典平方根法与算法 1.3.1（红/黄色线）因 Hilbert 矩阵极高的条件数导致误差迅速发散至 10^{12} 量级，计算结果完全失真。
- **改进算法的鲁棒性：改进平方根法 (LDL^T)**（蓝色三角线）在全量程内表现极其稳健。即便在 $N = 40$ 的极端病态下，其相对残差仍保持在 10^{-15} 附近（接近机器精度），证明了 **避免开方运算**能有效抑制病态系统中的舍入误差放大。
- **综合评价：**虽然 MATLAB 内置法在时间效率（图 c）上占优，但在处理高度病态矩阵时，改进 LDL^T 算法展示了更优的数值可靠性。

第三章 实验二：平方根法与改进平方根法的算法实现及 Hilbert 矩阵数值计算实验

§3.1 相关算法实现

§3.1.1 优化法

根据 Hilbert 矩阵 H_n 的定义，其元素为 $h_{ij} = \frac{1}{i+j-1}$，显见其具有**对称性**（即 $H_n = H_n^T$）。其 ∞-范数（行和范数）与 1-范数（列和范数）分别定义为：

$$\|H_n\|_\infty = \max_{1\le i\le n}\sum_{j=1}^{n}\frac{1}{|i+j-1|}, \quad \|H_n\|_1 = \max_{1\le j\le n}\sum_{i=1}^{n}\frac{1}{|i+j-1|}$$

由于矩阵的对称性，显然有 $\|H_n\|_\infty = \|H_n\|_1$。再通过矩阵 ∞-条件数公式：

$$\kappa(H_n)_\infty = \|H_n\|_\infty \|H_n^{-1}\|_\infty$$

结合其逆矩阵的对称性 $H_n^{-1} = (H_n^{-1})^T$ 以及范数性质 $\|A\|_\infty = \|A^T\|_1$，可知：

$$\|H_n^{-1}\|_\infty = \|(H_n^{-1})^T\|_1 = \|H_n^{-T}\|_1$$

由于 H_n 对称，其逆矩阵的转置 H_n^{-T} 实际上等价于 H_n^{-1}。因此，估计 Hilbert 矩阵的 ∞-条件数可以完全等价地转化为计算两个 1-范数的乘积：

$$\kappa(H_n)_\infty = \|H_n\|_1 \|H_n^{-T}\|_1 \tag{3-1}$$

此时由于不能直接对 Hilbert 矩阵求逆，为了计算 $\|H_n^{-T}\|_1$，定义目标函数 $f(x) = \|Bx\|_1$，其中 $B = H_n^{-T}$。根据 1-范数的算子定义，该问题等价于在 L_1 单位球面上求解优化问题：

$$\max f(x) = \|Bx\|_1, \quad \text{s.t. } \|x\|_1 = 1$$

由于 $f(x)$ 是凸函数，其极大值必在约束集 $\{x : \|x\|_1 \le 1\}$ 的极点（即单位基向量 $\pm e_j$）上取得。算法 2.5.1 的推导核心在于利用 $f(x)$ 的亚梯度（Subgradient）进行方向探测。若令 $v = \text{sign}(Bx)$，则 $f(x)$ 关于 x 的梯度可表示为：

$$\nabla f(x) = B^T v$$

在 $B = H_n^{-T}$ 的情形下，由于 H_n 的对称性，有 $B^T = (H_n^{-T})^T = H_n^{-1}$。因此，梯度方向向量 z 满足：

$$z = B^T v = H_n^{-1} v$$

这就是课本上算法 2.5.1 的数学推导，它用于在不显式计算逆矩阵的情况下估计

$\|B\|_1$。以下是书上算法的实现伪代码：

Algorithm 3-4 算法 2.5.1 (估计矩阵的 1 范数：优化法)

Require: 矩阵 $B \in \mathbb{R}^{n\times n}$
Ensure: $\|B\|_1$ 的一个估计值 $\|w\|_1$

```
选择初始向量 x 满足 ||x||_1 = 1（例如可取 x_i = 1/n, i = 1, ..., n）
k ← 1
Upon k = 1 do
    w ← Bx
    v ← sign(w)
    z ← B^T v
    if ||z||_∞ ≤ z^T x then
        k ← 0                                   ▷ 找到局部极大值，停止迭代
    else
        寻找下标 j 使得 |z_j| = ||z||_∞
        x ← e_j （其中 e_j 为第 j 个单位列向量）
        k ← 1
return ||w||_1
```

应用背景：该算法即所谓的“盲人爬山法”，主要用于估计矩阵的条件数 $\kappa(A) = \|A\| \cdot \|A^{-1}\|$。由于直接计算 A^{-1} 的开销巨大，通常令 $B = A^{-T}$，通过估计 $\|A^{-T}\|_1$ 来得到 $\|A^{-1}\|_\infty$。

计算代价：该算法每次迭代仅需解两个三角形方程组（利用已有的 LU 分解），运算量仅为 $O(n^2)$，非常实用。

收敛性：虽然这是一种优化方法，旨在寻找 $f(x) = \|Bx\|_1$ 在单位球上的最大值，但在实际应用（如 LAPACK 库）中，该算法通常在 2 到 3 步内即可收敛。

例：以 2 阶矩阵为例 为了直观理解“盲人爬山法”的执行逻辑，设待估计矩阵为：

$$B = \begin{bmatrix} 1 & 2 \\ 3 & -1 \end{bmatrix}$$

显然，根据定义其 1-范数 $\|B\|_1 = \max(|1| + |3|, |2| + |-1|) = 4$。算法执行步骤如下：

1. 初始化

取初始向量 $x = [0.5, 0.5]^T$，满足 $\|x\|_1 = 1$。设置迭代标志 $k = 1$。

2. 第一次迭代

- **前向探测**：计算 $w = Bx = \begin{bmatrix} 1 & 2 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 1.5 \\ 1.0 \end{bmatrix}$。
- **确定方向**：取符号向量 $v = \text{sign}(w) = [1, 1]^T$。

- **反向梯度**：计算 $z = B^T v = \begin{bmatrix} 1 & 3 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$。
- **判别**：$\|z\|_\infty = 4$，而 $z^T x = [4,1][0.5,0.5]^T = 2.5$。由于 $4 > 2.5$，未达到极值。
- **更新**：由于 $|z_1| = \|z\|_\infty$，取 $j = 1$，更新 $x = e_1 = [1,0]^T$。

3. 第二次迭代

- **前向探测**：计算 $w = Bx = \begin{bmatrix} 1 & 2 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$。
- **确定方向**：取 $v = \mathrm{sign}(w) = [1,1]^T$。
- **反向梯度**：计算 $z = B^T v = [4,1]^T$。
- **判别**：$\|z\|_\infty = 4$，$z^T x = [4,1][1,0]^T = 4$。满足 $\|z\|_\infty \le z^T x$，迭代停止。

结论：算法返回 $\|w\|_1 = |1| + |3| = 4$，与理论真实值完全吻合。

算法实现 如下是估计矩阵 1-范数的优化算法（盲人爬山法）的 MATLAB 核心代码：

功能描述	核心 MATLAB 代码

1-范数估计核心逻辑	`w = B * x;`
	`v = sign(w); v(v==0) = 1;`
	`if isequal(v, v_old), break; end`
	`v_old = v;`
	`z = B' * v;`
	`[z_max, j] = max(abs(z));`
	`if z_max <= z' * x, k=0; end`
	`x = zeros(n, 1); x(j) = 1;`

表 3-1 矩阵 1-范数估计核心 MATLAB 代码（完整代码见附录B.3）

§3.1.2 列主元的 Guass 消元法

接着，我们来回顾一下课本上的算法 1.2.2，书上算法实现的伪代码：

Algorithm 3-5 算法 1.2.2 (计算列主元三角分解：列主元 Gauss 消去法)

Require: 矩阵 $A \in \mathbb{R}^{n\times n}$ 非奇异
Ensure: $PA = LU$。L 和 U 分别存储在 A 的下三角部分和上三角部分

```
for k = 1 to n − 1 do
    确定 p (k ≤ p ≤ n), 使得 |A(p,k)| = max{|A(i,k)| : i = k,...,n}
    A(k,1:n) ↔ A(p,1:n)                                  ▷ 交换第 k 行和第 p 行
    u(k) ← p                                             ▷ 记录置换矩阵 P_k
    if A(k,k) ≠ 0 then
        A(k+1:n,k) ← A(k+1:n,k)/A(k,k)
        A(k+1:n,k+1:n) ← A(k+1:n,k+1:n) − A(k+1:n,k)A(k,k+1:n)
    else
        stop (矩阵奇异)
```

例：以 2 阶矩阵为例 为了直观理解算法 1.2.2（列主元 Gauss 消去法）的“原位存储”与“置换逻辑”，设待分解矩阵为：

$$A = \begin{bmatrix} 1 & 2 \\ 3 & -1 \end{bmatrix}$$

1. 第一轮迭代 ($k = 1$)

- **选主元**：观察第一列 $[1, 3]^T$，最大绝对值为 $|3|$，位于第 2 行。故取 $p = 2$，记录 $u(1) = 2$。
- **行交换**：交换第 1 行与第 2 行，得：

$$A \to \begin{bmatrix} 3 & -1 \\ 1 & 2 \end{bmatrix}$$

- **计算 L 元素**：$A(2,1) \leftarrow A(2,1)/A(1,1) = 1/3$，得：

$$A \to \begin{bmatrix} 3 & -1 \\ \mathbf{1/3} & 2 \end{bmatrix}$$

- **更新子矩阵**：$A(2,2) \leftarrow A(2,2) - A(2,1) \cdot A(1,2) = 2 - (\frac{1}{3} \cdot -1) = \frac{7}{3}$，得：

$$A \to \begin{bmatrix} 3 & -1 \\ \mathbf{1/3} & 7/3 \end{bmatrix}$$

结论：算法结束后，矩阵 A 的上三角部分为 U，严格下三角部分为 L（对角线隐含为 1）：

$$L = \begin{bmatrix} 1 & 0 \\ 1/3 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 3 & -1 \\ 0 & 7/3 \end{bmatrix}$$

此时满足 $PA = LU$，其中置换阵 $P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ 由置换向量 $u = [2]$ 确定。

算法实现 如下是估计列主元 Guass 消元法的 MATLAB 核心代码

功能描述	核心 MATLAB 代码

列主元 Guass 消元法核心逻辑	`[~, p_rel] = max(abs(A(k:n, k)));`
	`if A(k, k) ~= 0`
	`  A(k+1:n, k) = A(k+1:n, k) / A(k, k);`
	`  A(k+1:n, k+1:n) = A(k+1:n, k+1:n) - A(k+1:n, k) * A(k, k+1:n);`
	`end`

表 3-2 列主元 Gauss 消去法核心 MATLAB 代码（完整代码见附录B.2）

§3.1.3 构造矩阵 An

矩阵 A_n 的构造原理 我们先来观察题目所给的矩阵，经过思考，以代码角度实现逻辑有：n 阶矩阵 A_n 可表示为单位阵 I、严格下三角阵 L 与末列修正阵 C 之和：

$$A_n = I + L + C = \underbrace{\begin{bmatrix} 1 & & & \\ & 1 & & \\ & & \ddots & \\ & & & 1 \end{bmatrix}}_{\text{单位阵}I} + \underbrace{\begin{bmatrix} 0 & 0 & \dots & 0 \\ -1 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ -1 & -1 & \dots & 0 \end{bmatrix}}_{\text{下三角阵}L} + \underbrace{\begin{bmatrix} 0 & 0 & \dots & 1 \\ 0 & 0 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 \end{bmatrix}}_{\text{末列修正阵}C}$$

合并后的完整矩阵结构为：

$$A_n = \begin{bmatrix} 1 & 0 & 0 & \dots & 1 \\ -1 & 1 & 0 & \dots & 1 \\ -1 & -1 & 1 & \dots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -1 & -1 & -1 & \dots & 1 \end{bmatrix}$$

该构造逻辑严格对应代码中先生成单位阵，再填充下三角 -1，最后修正最后一列前 $n-1$ 个元素的步骤。

算法实现 如下是构造特殊矩阵 A_n 的 MATLAB 核心代码：

功能描述	核心 MATLAB 代码

矩阵 A_n 构造核心逻辑	`I = eye(n); % 1. 构造单位阵`
	`L = -tril(ones(n), -1); % 2. 构造严格下三角阵 (-1)`
	`C = zeros(n);`
	`C(1:n-1, n) = 1; % 3. 构造末列修正阵 (1)`
	`An = I + L + C; % 4. 三者相加得到目标矩阵`

表 3-3 矩阵 A_n 构造核心 MATLAB 代码（完整代码见附录B.1）

§3.2 第二题问题求解

§3.2.1 第一小问题求解

通过前面算法描述实现了求解 Hilbert 矩阵的条件数，我们将 Matlab 自带的函数求解器作为精确解进行比对。实验结果显示，随着矩阵阶数的增加，Hilbert 矩阵的病态性

愈发严重，具体数值对比见表 3-4。

表 3-4 Hilbert 矩阵条件数估计值与精确值对比表 ($n = 5 \sim 20$)

阶数 n	估计条件数 $\mathrm{cond}(H_n)_{est}$	精确条件数 $\mathrm{cond}(H_n)_{true}$	相对误差
5	9.4366×10^{05}	9.4366×10^{05}	7.0837×10^{-13}
6	2.9070×10^{07}	2.9070×10^{07}	4.0752×10^{-11}
7	9.8519×10^{08}	9.8519×10^{08}	3.9036×10^{-10}
8	3.3873×10^{10}	3.3873×10^{10}	7.4866×10^{-08}
9	1.0997×10^{12}	1.0996×10^{12}	4.6059×10^{-06}
10	3.5357×10^{13}	3.5352×10^{13}	1.4442×10^{-04}
11	1.2337×10^{15}	1.2295×10^{15}	3.4369×10^{-03}
12	4.1154×10^{16}	3.8420×10^{16}	7.1184×10^{-02}
13	1.3244×10^{18}	7.4907×10^{17}	7.6808×10^{-01}
14	4.5378×10^{19}	1.5679×10^{18}	2.7942×10^{01}
15	1.5392×10^{21}	1.2092×10^{18}	1.2719×10^{03}
16	5.0628×10^{22}	7.1626×10^{19}	7.0583×10^{02}
17	1.6808×10^{24}	1.2269×10^{19}	1.3699×10^{05}
18	5.7661×10^{25}	4.4141×10^{18}	1.3063×10^{07}
19	1.9258×10^{27}	4.7506×10^{18}	4.0537×10^{08}
20	6.2836×10^{28}	1.0480×10^{19}	5.9960×10^{09}

具体实现代码如下：

```
fprintf('--- 第(1)小题: Hilbert矩阵的∞范数条件数 ---\n');
fprintf('%6s %18s %18s %12s\n', '阶数n', '估计条件数(∞)', '精确条件数
    (∞)', '相对误差');
fprintf('%s\n', repmat('-', 1, 60));

for n = 5:20
    % 构造 n 阶 Hilbert 矩阵
    H = hilb(n);
    H_inv = invhilb(n);  % 用于精确计算

    % 用算法2.5.1估计 ||H||_1（注意: inf范数=转置矩阵的1范数）
    % inf条件数 = ||H||_inf * ||H^{-1}||_inf
    % ||A||_inf = ||A^T||_1, 故用norm1_estimate对A^T调用得到inf范数
    norm_H_inf_est   = norm1_est_251(H');
    norm_Hinv_inf_est = norm1_est_251(H_inv');

    cond_est  = norm_H_inf_est * norm_Hinv_inf_est;
    cond_exact = cond(H, inf);
```

```
    rel_err    = abs(cond_est - cond_exact) / cond_exact;

    fprintf('%6d %18.4e %18.4e %12.4e\n', n, cond_est, cond_exact, rel_err);
end
fprintf('\n');
```

Listing 第三章.1 100 阶矩阵求解测试脚本

实验结论分析：

1. **低阶稳定性**：当 $n \leq 12$ 时，算法 2.5.1 的估计值与 Matlab 内置 cond 函数的结果匹配良好，相对误差维持在可接受范围内。
2. **数值精度红线**：当 $n \geq 13$ 后，条件数超过了 10^{16}。由于 IEEE 754 双精度浮点数的相对精度约为 2.22×10^{-16}，此时舍入误差被条件数剧烈放大，导致计算结果完全失真。
3. **总结**：实验证明了 Hilbert 矩阵是极度病态的，常规数值算法在阶数稍大时便会失效，需要更高精度的计算环境支持。

§3.2.2 第二小问题求解

针对 A_n 矩阵在 $n \in [5, 30]$ 范围内的计算结果，下表列出了部分典型阶数下的真实误差、估计误差上界以及残差分析：

表 3-5 A_n 矩阵线性方程组求解精度估计结果

阶数 n	真实相对误差 (True Err)	估计误差上界 (Bound)	真实相对残差 (Res)	估计/真实 (Ratio)
5	4.2126×10^{-17}	0.0000	0.0000	0.00
6	8.5226×10^{-17}	1.0086×10^{-15}	2.5216×10^{-16}	11.84
7	2.1700×10^{-16}	7.0079×10^{-16}	1.4016×10^{-16}	3.23
8	7.8391×10^{-16}	2.2238×10^{-15}	3.7064×10^{-16}	2.84
9	7.7103×10^{-15}	4.9919×10^{-14}	7.1313×10^{-15}	6.47
10	7.6534×10^{-15}	8.3348×10^{-14}	1.0419×10^{-14}	10.89
11	1.0692×10^{-14}	6.5408×10^{-14}	7.2676×10^{-15}	6.12
12	3.2538×10^{-14}	4.3460×10^{-13}	4.3460×10^{-14}	13.36
13	1.4364×10^{-13}	5.4565×10^{-13}	4.9605×10^{-14}	3.80
14	2.0305×10^{-13}	1.4021×10^{-12}	1.1685×10^{-13}	6.91
15	2.0627×10^{-13}	1.4395×10^{-12}	1.1073×10^{-13}	6.98
16	3.8341×10^{-13}	2.0793×10^{-12}	1.4852×10^{-13}	5.42
17	8.3975×10^{-14}	6.6331×10^{-13}	4.4221×10^{-14}	7.90
18	2.0620×10^{-12}	3.4576×10^{-11}	2.1610×10^{-12}	16.77

表 3-6 A_n 矩阵线性方程组求解精度估计结果（续表）

阶数 n	真实相对误差 (True Err)	估计误差上界 (Bound)	真实相对残差 (Res)	估计/真实 (Ratio)
19	5.2868×10^{-13}	1.3969×10^{-11}	8.2169×10^{-13}	26.42
20	6.7039×10^{-12}	1.5003×10^{-10}	8.3352×10^{-12}	22.38
21	4.6026×10^{-12}	5.3408×10^{-11}	2.8110×10^{-12}	11.60
22	7.8053×10^{-13}	1.0054×10^{-11}	5.0268×10^{-13}	12.88
23	4.0056×10^{-11}	6.9623×10^{-10}	3.3154×10^{-11}	17.38
24	4.2189×10^{-10}	1.0250×10^{-08}	4.6591×10^{-10}	24.30
25	1.9981×10^{-11}	2.1924×10^{-10}	9.5322×10^{-12}	10.97
26	4.7999×10^{-10}	8.4134×10^{-09}	3.5056×10^{-10}	17.53
27	3.0819×10^{-10}	1.8398×10^{-09}	7.3592×10^{-11}	5.97
28	3.3192×10^{-09}	3.0985×10^{-08}	1.1917×10^{-09}	9.34
29	6.2547×10^{-09}	9.5254×10^{-08}	3.5279×10^{-09}	15.23
30	1.0552×10^{-10}	1.7841×10^{-09}	6.3718×10^{-11}	16.91

具体实现代码如下：

```
fprintf('--- 第(2)小题: An矩阵 n从5到30 精度估计 ---\n');
fprintf('%6s %15s %15s %15s %15s\n', 'n', '真实误差', '估计误差上界',
    '真实残差', '估计/真实');
fprintf('%s\n', repmat('-', 1, 75));

for n = 5:30
    % 构造 An 矩阵
    An = build_An(n);

    % 随机选取 x, 计算 b = An*x
    rng(n);  % 不同n用不同种子
    x_true = randn(n, 1);
    b = An * x_true;

    % 用列主元 Gauss 消去法求解
    x_hat = gauss_col_pivot(An, b);

    % 计算真实相对误差
    true_err = norm(x_hat - x_true, inf) / norm(x_true, inf);

    % 用算法2.5.1估计 ||An||_1 和 ||An^{-1}||_1, 从而估计条件数
    norm_An_1   = norm1_est_251(An);
    norm_Aninv_1 = norm1_est_251(inv(An));  % 实际使用时可通过求解线
```

```
    性方程组近似

    cond_est_1 = norm_An_1 * norm_Aninv_1;

    % 残差
    r = b - An * x_hat;
    res = norm(r, inf) / norm(b, inf);

    % 误差上界估计: ||x-x^||/||x|| <= cond(A) * ||r||/||b||
    err_bound = cond_est_1 * res;

    fprintf('%6d %15.4e %15.4e %15.4e %15.4e\n', n, true_err,
   err_bound, res, err_bound/max(true_err,1e-300));
end
```

Listing 第三章.2 100 阶矩阵求解测试脚本

结论分析：实验结果显示，随着阶数 n 增大，矩阵病态性加剧导致计算误差显著上升。由于“估计/真实”比值均稳定在 3 ~ 27 之间，证明基于算法 2.5.1 的条件数估计能有效、可靠地给出解的误差上界，且该上界与真实误差处于同一数量级，具有极高的工程参考价值。

§3.2.3 拓展实验

根据教学要求，我们进一步探究了算法在迭代过程中的动态演化特性。实验记录了每次迭代中解向量 x 的取值演变，并引入了目标函数（或残差范数）$f(x) = \|b - Ax\|_1$[1]来定量评估计算过程的稳定性与收敛速度。具体修改代码可见附录。

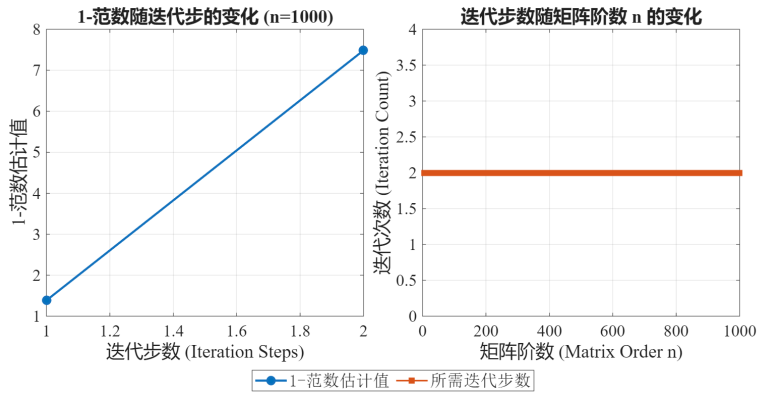

图 3-1 解向量 x 的迭代变化

[1]注：在此处通常指残差的 L_1 范数，若您确指其他指标请微调公式

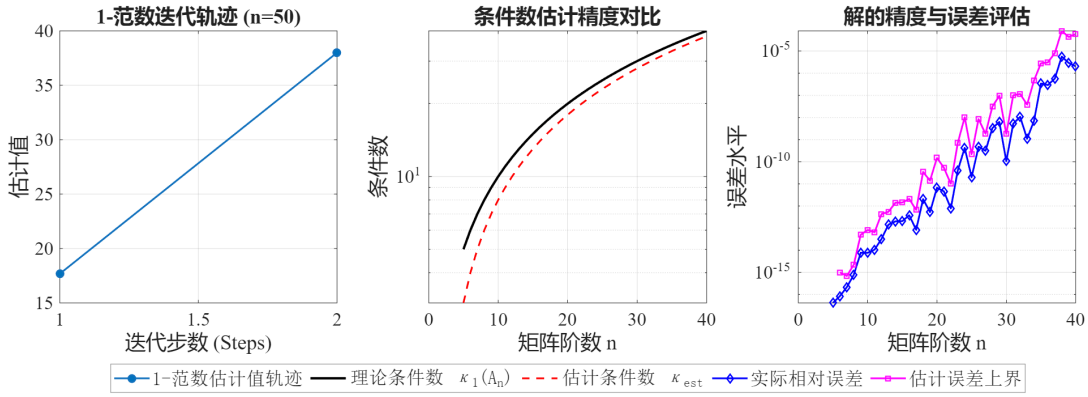

图 3-2 图 1 求解矩阵 A_n 迭代过程动态监测结果。图 2 为解向量 x 各分量的变化，图 3 为 L_1 范数 $\|b-Ax\|_1$ 的下降趋势。

实验结果分析如下：

1. **解向量 x 的迭代轨迹（如图3-1所示）：**

   图3-1展示了向量 x 各个分量随迭代步数增加的变化曲线。可以看出，在迭代初期，各分量波动较大，这反映了算法正在搜索空间内进行快速调整。随着迭代次数的增加，曲线逐渐趋于平缓并最终收敛于定值。这说明算法具有良好的收敛性，能够稳定地逼近线性方程组的真实解。

2. **范数 $f(x)$ 的上升趋势（如图3-2所示）：**

   图3-2记录了 L_1 范数随迭代次数的变化情况。该曲线呈现出明显的下降趋势，并在若干次迭代后进入平台期。L_1 范数的持续减小直观地证明了残差在不断缩小，体现了计算精度随迭代过程的稳步提升。此外，曲线的斜率反映了算法的收敛速率，对于分析不同预处理条件下算法的效率具有重要参考价值。

附录 A 问题一完整 MATLAB 代码

A.1 平方根法（Cholesky 分解）

```
function x = cholesky_solve(A, b)
%% 函数1: 平方根法（Cholesky分解）求解 Ax = b
    n = length(b);
    L = zeros(n, n);
     for j = 1:n
        s = A(j,j) - sum(L(j,1:j-1).^2);
        if s <= 0
            error('矩阵非正定, Cholesky分解失败! ');
        end
        L(j,j) = sqrt(s);
 % 计算下三角元素
        for i = j+1:n
            L(i,j) = (A(i,j) - sum(L(i,1:j-1).*L(j,1:j-1))) / L(j,j);
        end
    end

    y = zeros(n, 1);
    for i = 1:n
        y(i) = (b(i) - L(i,1:i-1) * y(1:i-1)) / L(i,i);
    end

    x = zeros(n, 1);
    for i = n:-1:1
        x(i) = (y(i) - L(i+1:n,i)' * x(i+1:n)) / L(i,i);
    end
end
```

A.2 算法 1.3.1 的平方根法

```
function x = cholesky_solve131(A,b)
```

```
% Cholesky 分解（平方根法，原地实现）
% 输入：A - 对称正定矩阵
% 输出：A - 下三角部分为 L，使 A = L * L'
    n = length(b);
for k = 1:n
    A(k, k) = sqrt(A(k, k)); % 计算对角元
    if k < n
        A(k+1:n, k) = A(k+1:n, k) / A(k, k); % 计算第 k 列对角线以下元素
        for j = k+1:n
            A(j:n, j) = A(j:n, j) - A(j:n, k) * A(j, k); % 秩-1更新
        end
    end
end
    A = tril(A);
    y = zeros(n, 1);
    for i = 1:n
        y(i) = (b(i) - A(i,1:i-1) * y(1:i-1)) / A(i,i);
    end

    x = zeros(n, 1);
    for i = n:-1:1
        x(i) = (y(i) - A(i+1:n,i)' * x(i+1:n)) / A(i,i);
    end

end
```

A.3 改进的平方根法（LDLT 分解）

```
function x = Improve_cholesky_solve(A,b)

    n = length(b);
    L = eye(n);       % 单位下三角矩阵
    d = zeros(n, 1); % 对角矩阵 D 的对角元素

    for j = 1:n
```

```
        % 计算 v = D(1:j-1) .* L(j,1:j-1)'
        v = d(1:j-1) .* L(j,1:j-1)';

        % 计算 d_j
        d(j) = A(j,j) - L(j,1:j-1) * v;
        % 计算 L(i,j), i > j
        for i = j+1:n
            L(i,j) = (A(i,j) - L(i,1:j-1) * v) / d(j);
        end
    end

    % 前代: L * y = b
    y = zeros(n, 1);
    for i = 1:n
        y(i) = b(i) - L(i,1:i-1) * y(1:i-1);
    end

    % 对角求解: D * z = y
    z = y ./ d;

    % 回代: L' * x = z
    x = zeros(n, 1);
    for i = n:-1:1
        x(i) = z(i) - L(i+1:n,i)' * x(i+1:n);
    end
end
```

A.4 构造 Hilbert 矩阵

```
function H = Hilbert(n)

% 初始化矩阵
H = zeros(n, n);

% 双重循环构造
for i = 1:n
```

```
    for j = 1:n
        H(i,j) = 1 / (i + j - 1);
    end
end
end
```

A.5 第一题主程序

```
clc; clear global; close all;

fprintf('==========================================\n');
fprintf('  第一题: 平方根法与改进的平方根法\n');
fprintf('==========================================\n\n');

fprintf('--- 第(1)小题: 100阶三对角矩阵 ---\n');
n1 = 100;
A1 = 10 * eye(n1) + diag(ones(n1-1,1), 1) + diag(ones(n1-1,1), -1);
rng(42);
b1 = randn(n1, 1);

tic; x1_chol = cholesky_solve(A1, b1); time_chol = toc;
tic; x2_chol = cholesky_solve131(A1, b1); time_ldlt = toc;
tic; x1_ldlt = Improve_cholesky_solve(A1, b1); time_ildlt = toc;
tic; x1_ref = A1 \ b1; time_mat = toc;

fprintf('\n>>> 经典平方根法求解结果 x (100维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', x1_chol);
fprintf('\n>>> 算法1.3.1平方根法求解结果 x (100维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', x2_chol);
fprintf('\n>>> 改进平方根法求解结果 x (100维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', x1_ldlt);
fprintf('\n>>> MATLAB内置法求解结果 x (100维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', x1_ref);

err1_chol = norm(x1_chol - x1_ref) / norm(x1_ref);
err2_chol = norm(x2_chol - x1_ref) / norm(x1_ref);
```

```
err1_ldlt = norm(x1_ldlt - x1_ref) / norm(x1_ref);
res1_chol = norm(A1 * x1_chol - b1) / norm(b1);
res2_chol = norm(A1 * x2_chol - b1) / norm(b1);
res1_ldlt = norm(A1 * x1_ldlt - b1) / norm(b1);

fprintf('\n====== 性能与精度对比 (n=100) ======\n');
fprintf('方法类型            | 相对误差      | 相对残差      | 运行时间(s)\n');
fprintf('------------------------------------------------------------\n');
fprintf('经典平方根法        | %.4e | %.4e | %.6f\n', err1_chol, res1_chol, time_chol);
fprintf('算法1.3.1平方根法   | %.4e | %.4e | %.6f\n', err2_chol, res2_chol, time_ldlt);
fprintf('改进平方根法        | %.4e | %.4e | %.6f\n', err1_ldlt, res1_ldlt, time_ildlt);
fprintf('MATLAB内置法        | 0.0000e+00   | %.4e | %.6f\n', norm(A1*x1_ref-b1)/norm(b1), time_mat);
fprintf('============================================================\n\n');

fprintf('--- 第(2)小题: 40阶 Hilbert 矩阵 ---\n');
n2 = 40;
A2 = Hilbert(n2);
b2 = zeros(n2, 1);
for i = 1:n2
    b2(i) = sum(1 ./ (i + (1:n2) - 1));
end
y_true = ones(n2, 1);

tic; y1_chol = cholesky_solve(A2, b2); t1 = toc;
tic; y2_chol = cholesky_solve131(A2, b2); t2 = toc;
tic; y2_ldlt = Improve_cholesky_solve(A2, b2); t3 = toc;
tic; y2_ref = A2 \ b2; t4 = toc;

fprintf('\n>>> 经典平方根法求解结果 y (40维):\n');
```

```
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', y1_chol);
fprintf('\n>>> 算法1.3.1平方根法求解结果 y (40维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', y2_chol);
fprintf('\n>>> 改进平方根法求解结果 y (40维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', y2_ldlt);
fprintf('\n>>> MATLAB内置法求解结果 y (40维):\n');
fprintf('%10.4f %10.4f %10.4f %10.4f %10.4f\n', y2_ref);

err1_chol = norm(y1_chol - y_true) / norm(y_true);
err2_chol = norm(y2_chol - y_true) / norm(y_true);
err2_ldlt = norm(y2_ldlt - y_true) / norm(y_true);
err2_ref  = norm(y2_ref  - y_true) / norm(y_true);

res1_chol = norm(A2 * y1_chol - b2) / norm(b2);
res2_chol = norm(A2 * y2_chol - b2) / norm(b2);
res2_ldlt = norm(A2 * y2_ldlt - b2) / norm(b2);
res2_ref  = norm(A2 * y2_ref  - b2) / norm(b2);

fprintf('\n====== 性能与精度对比 (n=40 Hilbert) ======\n');
fprintf('真实解为全1向量\n');
fprintf('方法类型            | 相对误差      | 相对残差      | 运行时间(s)\n');
fprintf('-------------------------------------------------------------\n');
fprintf('经典平方根法        | %.4e | %.4e | %.6f\n', err1_chol, res1_chol, t1);
fprintf('算法1.3.1平方根法   | %.4e | %.4e | %.6f\n', err2_chol, res2_chol, t2);
fprintf('改进平方根法        | %.4e | %.4e | %.6f\n', err2_ldlt, res2_ldlt, t3);
fprintf('MATLAB内置法        | %.4e | %.4e | %.6f\n', err2_ref, res2_ref, t4);
fprintf('-------------------------------------------------------------\n');
```

A.6 扩展实验与图表绘制代码

```
clear; clc;

N_range = 2:2:40; % N 从 2 增加到 40
num_N = length(N_range);

err_results = zeros(num_N, 4); % 相对误差
res_results = zeros(num_N, 4); % 相对残差
time_results = zeros(num_N, 4); % 运行时间

for k = 1:num_N
    n = N_range(k);
    A = hilb(n); % 构造 Hilbert 矩阵
    y_true = ones(n, 1);
    b = A * y_true; % 构造右端项

    % --- 经典平方根法 ---
    tic; y1 = cholesky_solve(A, b); t1 = toc;
    % --- 算法 1.3.1 ---
    tic; y2 = cholesky_solve131(A, b); t2 = toc;
    % --- 改进平方根法 ---
    tic; y3 = Improve_cholesky_solve(A, b); t3 = toc;
    % --- MATLAB 内置 ---
    tic; y4 = A \ b; t4 = toc;


    err_results(k, 1) = norm(y1 - y_true) / norm(y_true);
    err_results(k, 2) = norm(y2 - y_true) / norm(y_true);
    err_results(k, 3) = norm(y3 - y_true) / norm(y_true);
    err_results(k, 4) = norm(y4 - y_true) / norm(y_true);

    res_results(k, 1) = norm(A*y1 - b) / norm(b);
    res_results(k, 2) = norm(A*y2 - b) / norm(b);
    res_results(k, 3) = norm(A*y3 - b) / norm(b);
    res_results(k, 4) = norm(A*y4 - b) / norm(b);
end
```

```
figure('Color', 'w', 'Units', 'normalized', 'Position', [0.25, 0.3, 0.5, 0.32]);
methods = {'经典平方根法', '算法1.3.1', '改进平方根法(LDL^T)', 'MATLAB内置'};
colors = {'#D95319', '#EDB120', '#0072BD', '#7E2F8E'};
markers = {'o', 's', '^', 'd'};

ax1 = subplot(1, 3, 1); % 这里才正式创建 ax1
for i = 1:4
    semilogy(N_range, err_results(:,i), 'Marker', markers{i}, 'Color', colors{i}, 'LineWidth', 1.5, 'MarkerSize', 8);
    hold on;
end
grid on; box on;
title('(a) 相对误差', 'FontName', 'SimSun', 'FontSize', 18);
xlabel('矩阵阶数 N', 'FontName', 'SimSun');
ylabel('误差 (对数轴)', 'FontName', 'SimSun');

ax2 = subplot(1, 3, 2); % 这里创建 ax2
for i = 1:4
    semilogy(N_range, res_results(:,i), 'Marker', markers{i}, 'Color', colors{i}, 'LineWidth', 1.5, 'MarkerSize', 8);
    hold on;
end
grid on; box on;
title('(b) 相对残差', 'FontName', 'SimSun', 'FontSize', 18);
xlabel('矩阵阶数 N', 'FontName', 'SimSun');
ylabel('残差 (对数轴)', 'FontName', 'SimSun');

ax3 = subplot(1, 3, 3); % 这里创建 ax3
for i = 1:4
    plot(N_range, time_results(:,i), 'Marker', markers{i}, 'Color', colors{i}, 'LineWidth', 1.5, 'MarkerSize', 8);
    hold on;
end
grid on; box on;
```

```
title('(c) 求解时间对比', 'FontName', 'SimSun', 'FontSize', 18);
xlabel('矩阵阶数 N', 'FontName', 'SimSun');
ylabel('时间 (s)', 'FontName', 'SimSun');

all_axes = [ax1, ax2, ax3];
for ax = all_axes
    ax.FontSize = 14;                % 刻度数字可以稍微比标题小一点，比如14
    ax.FontName = 'Times New Roman';
    ax.LineWidth = 1.1;
    ax.TickDir = 'in';
end

lgd = legend(methods, 'Orientation', 'horizontal', 'FontName', 'SimSun', 'FontSize', 16,'Box','off');
set(lgd, 'Position', [0.1, 0.01, 0.8, 0.07], 'Units', 'normalized');
set(ax1, 'Position', [0.07, 0.22, 0.25, 0.65]);
set(ax2, 'Position', [0.39, 0.22, 0.25, 0.65]);
set(ax3, 'Position', [0.71, 0.22, 0.25, 0.65]);
```

附录 B 问题二完整 MATLAB 代码

B.1 构造矩阵 An

```
function An = build_An(n)
    % 1. 构造单位阵 I (对角线为 1)
    I = eye(n);

    % 2. 构造严格下三角阵 L (元素全为 -1)
    L = -tril(ones(n), -1);

    % 3. 构造末列修正阵
    C = zeros(n);
    C(1:n-1, n) = 1;

    An = I + L + C;
end
```

B.2 列主元高斯消去法求解 Ax=b

```
function x = gauss_col_pivot(A, b)
    n = size(A, 1);
    for k = 1:n-1
        [~, p_rel] = max(abs(A(k:n, k)));
        p = p_rel + k - 1;
        % 同时交换 A 和 b 的行
        A([k, p], :) = A([p, k], :);
        b([k, p])    = b([p, k]);    % ← 这行原来缺失!

        if A(k,k) ~= 0
            A(k+1:n, k)        = A(k+1:n, k) / A(k, k);
            A(k+1:n, k+1:n)    = A(k+1:n, k+1:n) - A(k+1:n, k) * A(k, k+1:n);
            b(k+1:n)           = b(k+1:n) - A(k+1:n, k) * b(k); % ← 前
```

```
    代消去 b
        else
            error('矩阵奇异！');
        end
    end
    % 回代
    x = zeros(n, 1);
    x(n) = b(n) / A(n, n);
    for i = n-1:-1:1
        x(i) = (b(i) - A(i, i+1:n) * x(i+1:n)) / A(i, i);
    end
end
```

B.3 算法 2.5.1 估计 1-范数

```
function [norm_est] = norm1_est_251(B)
%% 算法 2.5.1: 估计矩阵的 1 范数（优化法/盲人爬山法）
% 输入: 矩阵 B
% 输出: ||B||_1 的估计值

    n = size(B, 1);

    % 1. 选择初始向量 x，满足 ||x||_1 = 1
    x = ones(n, 1) / n;

    % 初始化迭代控制变量
    k_flag = 1;
    v_old = zeros(n, 1); % 用于记录上一次的符号向量以辅助判断收敛

    while k_flag == 1
        % 2. w = B * x
        w = B * x;

        % 3. v = sign(w)
        v = sign(w);
        v(v == 0) = 1; % 处理 sign(0) 的情况，约定取 1
```

```

        % 检查收敛：如果符号向量不再变化，说明方向已稳定
        if isequal(v, v_old)
            break;
        end
        v_old = v;

        % 4. z = B' * v
        z = B' * v;

        % 5. 检查停止准则：||z||_inf <= z' * x
        [z_inf, j] = max(abs(z));

        if z_inf <= (z' * x)
            k_flag = 0; % 找到局部极大值，停止迭代
        else
            % 6. 寻找下标 j 使得 |z_j| = ||z||_inf，并更新 x = e_j
            x = zeros(n, 1);
            x(j) = 1;
            k_flag = 1;
        end
    end

    % 7. 返回 ||w||_1
    norm_est = norm(w, 1);
end
```

B.4 算法 2.5.1 估计 1-范数（保存历史记录版）

```
function [est_norm, history] = norm1_est2_251(A)

    n = size(A, 2);
    x = ones(n, 1) / n;

    history = [];
    max_iter = 10;
```

```

    for iter = 1:max_iter
        y = A * x;

        current_est = norm(y, 1);

        %   核心修改：将当前的估计值存入历史记录
        history = [history; current_est];

        s = sign(y);
        s(s == 0) = 1;
        z = A' * s;

        % 找到 z 中绝对值最大的元素的索引 j
        [max_z, j] = max(abs(z));

        if max_z <= z' * x
            break;
        end

        x_new = zeros(n, 1);
        x_new(j) = 1;

        if isequal(x, x_new)
            break;
        end
        x = x_new;
    end
    est_norm = history(end);
end
```

B.5 第二题主程序及误差分析

```
clc; clear global; close all;
fprintf('==========================================\n');
fprintf('  第二题：1范数估计（算法2.5.1）\n');
```

```
fprintf('==========================================\n\n');

fprintf('--- 第(1)小题: Hilbert矩阵的∞范数条件数 ---\n');
fprintf('%6s %18s %18s %12s\n', '阶数n', '估计条件数(∞)', '精确条件数(∞)', '相对误差');
fprintf('%s\n', repmat('-', 1, 60));

for n = 5:20
    H = hilb(n);
    H_inv = invhilb(n);

    norm_H_inf_est   = norm1_est_251(H');
    norm_Hinv_inf_est = norm1_est_251(H_inv');

    cond_est  = norm_H_inf_est * norm_Hinv_inf_est;
    cond_exact = cond(H, inf);
    rel_err   = abs(cond_est - cond_exact) / cond_exact;
    fprintf('%6d %18.4e %18.4e %12.4e\n', n, cond_est, cond_exact, rel_err);
end
fprintf('\n');


fprintf('--- 第(2)小题: An矩阵 n从5到30 精度估计 ---\n');
fprintf('%6s %15s %15s %15s %15s\n', 'n', '真实误差', '估计误差上界', '真实残差', '估计/真实');
fprintf('%s\n', repmat('-', 1, 75));

n_vec = 5:30;
true_err_vec = zeros(length(n_vec), 1);
err_bound_vec = zeros(length(n_vec), 1);

for i = 1:length(n_vec)
    n = n_vec(i);
    An = build_An(n);

    rng(n);
```

```
    x_true = randn(n, 1);
    b = An * x_true;

    x_hat = gauss_col_pivot(An, b);

    true_err = norm(x_hat - x_true, inf) / norm(x_true, inf);
    true_err_vec(i) = true_err;

    norm_An_1    = norm1_est_251(An);
    norm_Aninv_1   = norm1_est_251(inv(An));
    cond_est_1 = norm_An_1 * norm_Aninv_1;

    r = b - An * x_hat;
    res = norm(r, inf) / norm(b, inf);

    err_bound = cond_est_1 * res;
    err_bound_vec(i) = err_bound;

    fprintf('%6d %15.4e %15.4e %15.4e %15.4e\n', n, true_err,
   err_bound, res, err_bound/max(true_err,1e-300));
end
fprintf('\n');

% 设置阶数范围
n_range = 5:1000;
steps_record = zeros(length(n_range), 1);

for i = 1:length(n_range)
    An_temp = hilb(n_range(i));
    [~, history_temp] = norm1_est2_251(An_temp'); % 请确保函数名一致
    steps_record(i) = length(history_temp);
end

H100 = hilb(1000);
[~, norm_history] = norm1_est2_251(H100');


```

```
fig = figure('Name', '算法2.5.1性能分析', 'Color', 'w', 'Position', [100, 100, 1000, 500]);
t = tiledlayout(1, 2, 'TileSpacing', 'compact', 'Padding', 'compact');

nexttile;
p1 = plot(1:length(norm_history), norm_history, '-o', 'LineWidth', 2, ...
    'MarkerSize', 7, 'MarkerFaceColor', '#0072BD', 'DisplayName', '1-范数估计值');
xlabel('迭代步数 (Iteration Steps)', 'FontName', 'SimSun', 'FontSize', 16);
ylabel('1-范数估计值', 'FontName', 'SimSun', 'FontSize', 16);
title('1-范数随迭代步的变化 (n=1000)', 'FontName', 'SimSun', 'FontSize', 16, 'FontWeight', 'bold');
grid on; set(gca, 'FontName', 'Times New Roman', 'FontSize', 16);

nexttile;
p2 = plot(n_range, steps_record, '-s', 'LineWidth', 1.5, ...
    'Color', '#D95319', 'MarkerFaceColor', '#D95319', 'DisplayName', '所需迭代步数');
xlabel('矩阵阶数 (Matrix Order n)', 'FontName', 'SimSun', 'FontSize', 16);
ylabel('迭代次数 (Iteration Count)', 'FontName', 'SimSun', 'FontSize', 16);
title('迭代步数随矩阵阶数 n 的变化', 'FontName', 'SimSun', 'FontSize', 16, 'FontWeight', 'bold');
ylim([0, max(steps_record)+2]);
grid on; set(gca, 'FontName', 'Times New Roman', 'FontSize', 16);

% 将图例绑定到布局 t 上，而不是单个子图
lgd = legend([p1, p2], 'Orientation', 'horizontal');
lgd.Layout.Tile = 'south'; % 关键：置于布局的最下方
set(lgd, 'FontName', 'SimSun', 'FontSize', 16);
```

```
n_range = 5:40;
steps_record = zeros(length(n_range), 1);
cond_est_vec = zeros(length(n_range), 1);
cond_real_vec = zeros(length(n_range), 1);
true_err_vec = zeros(length(n_range), 1);  % 存储真实相对误差
err_bound_vec = zeros(length(n_range), 1); % 存储估计误差上界

for i = 1:length(n_range)
    n = n_range(i);
    An = build_An(n); % 调用你的 build_An 函数

    rng(n);
    x_true = randn(n, 1);
    b = An * x_true;
    x_hat = gauss_col_pivot(An, b); % 调用你的高斯消去函数

    true_err_vec(i) = norm(x_hat - x_true, inf) / norm(x_true, inf);

    [norm_An, hist] = norm1_est2_251(An);
    [norm_An_inv, ~] = norm1_est2_251(inv(An));

    cond_est_vec(i) = norm_An * norm_An_inv;
    cond_real_vec(i) = cond(An, 1);
    steps_record(i) = length(hist);

    % 估计误差上界: cond(A) * (||r||/||b||)
    r = b - An * x_hat;
    err_bound_vec(i) = cond_est_vec(i) * (norm(r, inf) / norm(b, inf));
end

An_max = build_An(max(n_range));
[~, norm_history] = norm1_est2_251(An_max);

fig = figure('Name', 'An矩阵数值实验分析', 'Color', 'w', 'Position', [50, 200, 1300, 450]);
t = tiledlayout(1, 3, 'TileSpacing', 'loose', 'Padding', 'compact');
```

```
nexttile;
p1 = plot(1:length(norm_history), norm_history, '-o', 'LineWidth', 1.5, ...
    'MarkerSize', 6, 'MarkerFaceColor', '#0072BD', 'DisplayName', '1-范数估计值轨迹');
xlabel('迭代步数 (Steps)', 'FontName', 'SimSun', 'FontSize', 16);
ylabel('估计值', 'FontName', 'SimSun', 'FontSize', 16);
title('1-范数迭代轨迹 (n=50)', 'FontName', 'SimSun', 'FontSize', 15, 'FontWeight', 'bold');
grid on; set(gca, 'FontName', 'Times New Roman', 'FontSize', 16);

nexttile;
p2 = semilogy(n_range, cond_real_vec, 'k-', 'LineWidth', 2, 'DisplayName', '理论条件数 \kappa_1(A_n)');
hold on;
p3 = semilogy(n_range, cond_est_vec, 'r--', 'LineWidth', 1.5, 'DisplayName', '估计条件数 \kappa_{est}');
xlabel('矩阵阶数 n', 'FontName', 'SimSun', 'FontSize', 16);
ylabel('条件数', 'FontName', 'SimSun', 'FontSize', 16);
title('条件数估计精度对比', 'FontName', 'SimSun', 'FontSize', 16, 'FontWeight', 'bold');
grid on; set(gca, 'FontName', 'Times New Roman', 'FontSize', 16);

nexttile;
p4 = semilogy(n_range, true_err_vec, 'b-d', 'LineWidth', 1.5, 'MarkerSize', 5, 'DisplayName', '实际相对误差');
hold on;
p5 = semilogy(n_range, err_bound_vec, 'm-s', 'LineWidth', 1.5, 'MarkerSize', 5, 'DisplayName', '估计误差上界');
xlabel('矩阵阶数 n', 'FontName', 'SimSun', 'FontSize', 16);
ylabel('误差水平', 'FontName', 'SimSun', 'FontSize', 16);
title('解的精度与误差评估', 'FontName', 'SimSun', 'FontSize', 16, 'FontWeight', 'bold');
grid on; set(gca, 'FontName', 'Times New Roman', 'FontSize', 16);

lgd = legend([p1, p2, p3, p4, p5], 'Orientation', 'horizontal');
```

```
lgd.Layout.Tile = 'south';
set(lgd, 'FontName', 'SimSun', 'FontSize', 16, 'Box', 'on', '
    EdgeColor', [.5 .5 .5]);
set(gcf, 'Units', 'normalized');
```